Rockchip Linux6.12 SDK Quick Start Guide

Document Identifier: RK-JC-YF-A67

Release Version: V1.0.1

Date: 2026-06-29

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily describes the basic usage of the Linux6.12 SDK, aiming to help developers quickly understand and use the Linux6.12 universal SDK development package.

Target Audience

This document (this guide) is mainly applicable to the following engineers:

Technical Support Engineers

Software Development Engineers

Chip System Support Status

Chip Name	U-Boot Version	Kernel Version	Buildroot Version	Yocto Version	Debian Version
RK3572	v5	6.12	2026.02	6.0	13
RK3576	v5	6.12	2026.02	6.0	13
RK3588	v5	6.12	2026.02	6.0	13

Revision Record

Date	Version	Author	Modification Description
2026-03-20	V0.0.1	Weiye Chen	Initial version
2026-03-30	V0.0.2	Weiye Chen	1. Remove Hardware Development Guide 2. Remove Chip Documentation 3. Correct certain file paths
2026-04-20	V0.1.0	Weiye Chen	Beta version
2026-06-20	V1.0.0	Weiye Chen	Release version
2026-06-29	V1.0.1	Weiye Chen	Updated updateEngine upgrade guide

Table of Contents

Rockchip Linux6.12 SDK Quick Start Guide

1. Precompiled SDK Images
2. Development Environment Setup
   - 2.1 Preparing the Development Environment
   - 2.2 Installing Libraries and Toolkits
     - 2.2.1 Checking the Host's Python Version
     - 2.2.2 Checking the Host's `make` Version
     - 2.2.3 Checking the Host's LZ4 Version
3. Docker Environment Setup
4. Software Development Guide
   - 4.1 Development Guide
   - 4.2 Buildroot Development Guide
   - 4.3 Debian Development Guide
   - 4.4 Third-Party OS Porting
   - 4.5 DPDK Development Guide
   - 4.6 Real-Time Linux Development Guide
   - 4.7 RKIPC Developer Guide
   - 4.8 Rockchip Linux updateEngine Upgrade Guide
     - 4.8.1 Recovery Mode OTA Upgrade
     - 4.8.2 Linux A/B Mode OTA Upgrade
     - 4.8.3 Differential Upgrade
     - 4.8.4 SD Card Boot Upgrade
   - 4.9 Rockchip Linux ROS2 Software Development Guide
     - 4.9.1 Build ROS2 Dependency Packages
     - 4.9.2 Prepare Docker Environment
     - 4.9.3 Cross-Compile ROS2 in Docker
     - 4.9.4 Package rootfs and Verify
   - 4.10 Rockchip IDC Software Development Guide
     - 4.10.1 Build Configuration
     - 4.10.2 Run Example
   - 4.11 RKNN Development Guide
     - 4.11.1 RKNPU2 Platforms (RK3576, RK3588, etc.)
     - 4.11.2 RKNPU3 Platforms (RK3572)
   - 4.12 RKAI Development Guide
     - 4.12.1 Build Configuration
     - 4.12.2 Model Files
     - 4.12.3 LLM Run Examples
     - 4.12.4 VLM Run Examples
   - 4.13 Software Update Log
5. SDK Project Directory Overview
6. Introduction to Cross-Compilation Toolchain for SDK
   - 6.1 U-Boot and Kernel Compilation Toolchain
   - 6.2 Buildroot Toolchain
     - 6.2.1 Configuring the Compilation Environment
     - 6.2.2 Packaging Toolchain
   - 6.3 Debian Toolchain
   - 6.4 Yocto Toolchain
7. SDK Compilation Instructions
   - 7.1 Viewing SDK Compilation Commands
   - 7.2 SDK Board-Level Configuration
     - 7.2.1 Chip Selection
     - 7.2.2 Board Configuration Selection

7.3 SDK Custom Configuration
7.4 Fully Automatic Compilation
7.5 Module Compilation
7.5.1 U-Boot Compilation
7.5.2 Kernel Compilation
7.5.3 Recovery Compilation
7.5.4 Buildroot Compilation
7.5.4.1 Buildroot Module Compilation
7.5.5 Debian Compilation
7.5.6 Yocto Compilation
7.6 Firmware Packaging
8. Flashing Instructions
8.1 Windows Flashing Instructions
8.2 Linux Flashing Instructions
8.3 System Partition Description

1. Precompiled SDK Images

Developers can bypass the process of compiling the entire operating system from source code by using the precompiled Linux SDK image. By directly flashing the precompiled image to the development board, they can quickly start development and perform related assessments and comparisons. This can reduce the waste of development time and cost due to compilation issues.

The firmware can be downloaded from the public address Click here for SDK firmware download.

Firmware path: **Universal Linux SDK Firmware** -> **Linux6.12**

For those who need to modify the SDK code or require a quick start, please refer to the following sections.

2. Development Environment Setup

2.1 Preparing the Development Environment

We recommend using a system with Ubuntu 22.04 or a higher version for compilation. Other Linux versions may require corresponding adjustments to the software packages. In addition to system requirements, there are other hardware and software requirements.

Hardware Requirements: 64-bit system with hard disk space greater than 40GB. If you are performing multiple builds, you will need more hard disk space.

Software Requirements: System with Ubuntu 22.04 or a higher version.

2.2 Installing Libraries and Toolkits

When developing for devices using the command line, you can install the necessary libraries and tools required for compiling the SDK by following these steps.

Use the following `apt-get` commands to install the libraries and tools needed for subsequent operations:

```
sudo apt-get update && sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool zstd expect expect-dev g++ patchelf chrpath gawk texinfo \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu \
libgmp-dev libmpc-dev bc python-is-python3 python3-setuptools swig \
python3-dev
```

> **Note:**
> The installation command is suitable for Ubuntu 22.04. For other versions, please use the corresponding installation command based on the package name. If you encounter errors during compilation, you can install the corresponding software packages based on the error messages. Specifically:
>
> - Python 3.6 or later is required. Ubuntu 22.04 installs Python 3.10 by default.
> - Make 4.0 or later is required. Ubuntu 22.04 installs Make 4.3 by default.
> - LZ4 1.7.3 or later is required. Ubuntu 22.04 installs LZ4 1.9.3 by default.
> - Compiling Yocto requires a VPN network.

2.2.1 Checking the Host's Python Version

The method to check the host's Python version is as follows:

- Check the host's Python version

```
$ python3 --version
Python 3.10.6
```

If Python 3.6 or later is not available, install the corresponding package for the host system version.

2.2.2 Checking the Host's `make` Version

The method to check the host's `make` version is as follows:

- Check the host's `make` version

```
$ make -v
GNU Make 4.3
Built for x86_64-pc-linux-gnu
```

If Make 4.0 or later is not available, install the corresponding package for the host system version.

2.2.3 Checking the Host's LZ4 Version

The method to check the host's LZ4 version is as follows:

- Check the host's LZ4 version

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
```

If LZ4 1.7.3 or later is not available, install the corresponding package for the host system version.

3. Docker Environment Setup

To assist developers in quickly completing the complex preparation of the development environment mentioned above, we also provide a cross-compiler Docker image for clients to rapidly verify, thereby reducing the time required to build the compilation environment.

Before using the Docker environment, you can refer to the following document for operations: `<SDK>/docs/en/Linux/Docker/Rockchip_Developer_Guide_Linux_Docker_Deploy_EN.pdf`.

The verified systems are as follows:

Distribution Version	Docker Version	Image Loading	Firmware Compilation
Ubuntu 22.04	24.0.5	pass	pass
Ubuntu 21.10	20.10.12	pass	pass
Ubuntu 21.04	20.10.7	pass	pass
Ubuntu 18.04	20.10.7	pass	pass
Fedora 35	20.10.12	pass	NR (not run)

The Docker image can be obtained from the URL Docker Image.

4. Software Development Guide

4.1 Development Guide

To assist development engineers in quickly becoming proficient with SDK development and debugging, the "Rockchip_Developer_Guide_Linux_Software_EN.pdf" is released along with the SDK. It can be obtained in the `docs/en` directory and will be continuously improved and updated.

4.2 Buildroot Development Guide

To assist development engineers in quickly getting started with the development and debugging of the Buildroot system, the development guide "Rockchip_Developer_Guide_Buildroot_EN.pdf" is released along with the SDK. It can be obtained in the `docs/en/Linux/System` directory and will be continuously improved and updated.

4.3 Debian Development Guide

To assist development engineers in quickly becoming proficient with the development and debugging of the Debian system, the SDK release includes the "Rockchip_Developer_Guide_Debian_EN.pdf" development guide, which can be obtained under `docs/en/Linux/System` and will be continuously improved and updated.

4.4 Third-Party OS Porting

To assist development engineers in quickly mastering the porting and adaptation of third-party operating systems, the SDK release includes the development guide titled "Rockchip_Developer_Guide_Third_Party_System_Adaptation_EN.pdf". It can be obtained in the `docs/en/Linux/System` directory and will be continuously improved and updated.

4.5 DPDK Development Guide

To assist development engineers in quickly mastering DPDK development and debugging, the SDK release includes the "Rockchip_Developer_Guide_Linux_DPDK_EN.pdf" and "Rockchip_Developer_Guide_Linux_GMAC_DPDK_EN.pdf" development guides, which can be obtained in the `docs/en/Linux/DPDK` and `docs/en/Common/GMAC` directories respectively, and will be continuously improved and updated.

4.6 Real-Time Linux Development Guide

As products increasingly demand real-time performance, standard Linux's real-time capabilities are no longer sufficient for many products. It is necessary to optimize standard Linux to enhance real-time performance, such as through the use of PREEMPT_RT and the Xenomai real-time system.

Below are the latency results from stress tests conducted using Buildroot, based on both PREEMPT_RT and Xenomai, using RK3572 as an example:

Stress Test:

```
stress-ng -c 8 --io 2 --vm 2 --vm-bytes 512M --timeout 1000000s
```

- PREEMPT_RT

```
root@rk3572-buildroot:/# cyclictest -c 0 -m -t 8 -a 0-7 -p 99 -D24H
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 13.67 13.67 13.60 14/325 5269

T: 0 ( 2552) P:99 I:1000 C:86399989 Min:     3 Act:    4 Avg:    6 Max:     103
T: 1 ( 2553) P:99 I:1500 C:57599990 Min:     3 Act:    7 Avg:    9 Max:     114
T: 2 ( 2554) P:99 I:2000 C:43199988 Min:     3 Act:    6 Avg:    8 Max:     108
T: 3 ( 2555) P:99 I:2500 C:34559987 Min:     3 Act:    6 Avg:    8 Max:     126
T: 4 ( 2556) P:99 I:3000 C:28799987 Min:     3 Act:    9 Avg:   11 Max:      79
```

```
T: 5 ( 2557) P:99 I:3500 C:24685700 Min:     3 Act:    5 Avg:   11 Max:     88
T: 6 ( 2558) P:99 I:4000 C:21599985 Min:     4 Act:   10 Avg:   11 Max:     98
T: 7 ( 2559) P:99 I:4500 C:19199984 Min:     4 Act:   12 Avg:   10 Max:    105
(Testing for 24 hours)

Isolate CPU4-5 (add "isolcpus=4-5 nohz_full=4-5" to kernel cmdline):
root@rk3572-buildroot:/# cyclictest -c 0 -m -t 2 -a 4-5 -p 99 -D24H
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 14.13 14.17 14.08 16/315 6997

T: 0 ( 3563) P:99 I:1000 C:86399998 Min:     3 Act:    3 Avg:    3 Max:     27
T: 1 ( 3564) P:99 I:1500 C:57599995 Min:     3 Act:    3 Avg:    3 Max:     19
(Testing for 24 hours)
```

- Xenomai Cobalt Mode

```
root@rk3572-buildroot:/# echo 0 > /proc/xenomai/latency
root@rk3572-buildroot:/# /usr/demo/cyclictest -c 0 -m -t 8 -a 0-7 -p 99 -D24H
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 13.09 13.12 13.09 13/261 7378

T: 0 ( 2547) P:99 I:1000 C:86399996 Min:     2 Act:    6 Avg:    6 Max:    107
T: 1 ( 2548) P:99 I:1500 C:57599997 Min:     2 Act:    5 Avg:    8 Max:    109
T: 2 ( 2549) P:99 I:2000 C:43199997 Min:     2 Act:    4 Avg:    9 Max:    123
T: 3 ( 2550) P:99 I:2500 C:34559996 Min:     2 Act:    4 Avg:    9 Max:    119
T: 4 ( 2551) P:99 I:3000 C:28799997 Min:     2 Act:    9 Avg:   14 Max:    113
T: 5 ( 2552) P:99 I:3500 C:24685711 Min:     3 Act:   11 Avg:   13 Max:    105
T: 6 ( 2553) P:99 I:4000 C:21599996 Min:     3 Act:   10 Avg:    9 Max:     91
T: 7 ( 2554) P:99 I:4500 C:19199996 Min:     4 Act:   13 Avg:   11 Max:    110
(Testing for 24 hours)

Isolate CPU4-5 (add "isolcpus=4-5 nohz_full=4-5" to kernel cmdline):
root@rk3572-buildroot:/# echo 0 > /proc/xenomai/latency
root@rk3572-buildroot:/# /usr/demo/cyclictest -c 0 -m -t 2 -a 4-5 -p 99 -D24H
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 13.38 13.47 13.52 12/231 5898

T: 0 ( 2444) P:99 I:1000 C:86399996 Min:     2 Act:    3 Avg:    3 Max:     34
T: 1 ( 2445) P:99 I:1500 C:57599997 Min:     2 Act:    3 Avg:    3 Max:     30
(Testing for 24 hours)
```

Optimization Notes:

1. The ddr.bin for RK3572 has been specifically optimized for real-time systems. In the U-Boot project, you need to use ./make.sh rk3572-rt to generate loader.bin, which packages the DDR initialization firmware with real-time configuration integrated.
2. The RK3572 is divided into 3 clusters, with each cluster's L2 cache being independent of one another. It is recommended to isolate CPU4-5 (Cluster1) or CPU6-7 (Cluster2) as real-time cores. The isolated CPUs will have higher priority when accessing DDR.

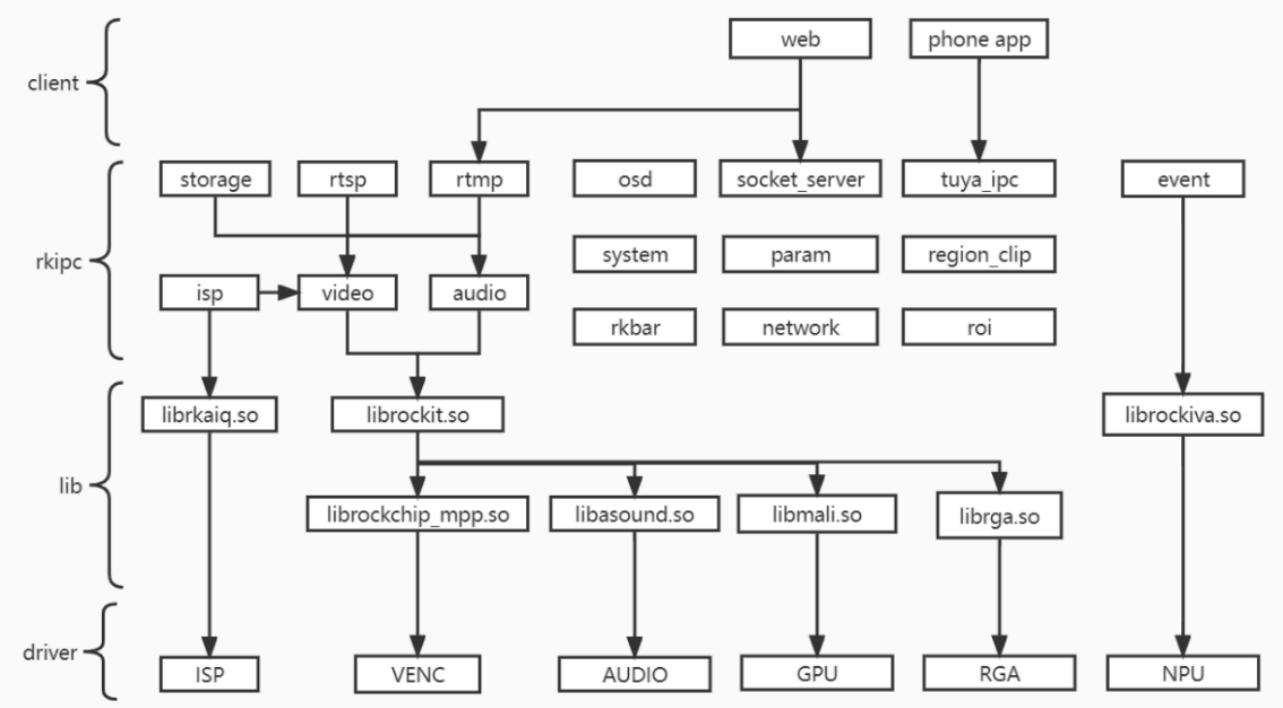
For more details, please refer to the development patch package and instructions in `docs/Patches/Real-Time-Performance/`.

Reference document for the real-time performance test data of Rockchip chips under different RT Linux implementations is as follows:

`docs/en/Common/PERF/Rockchip_Developer_Guide_Linux_RealTime_Performance_Test_Report_EN.pdf`

4.7 RKIPC Developer Guide

This guide targets IPC products on the RK platform, supporting web interfaces, RTSP/RTMP preview, and dynamic parameter adjustment. The overall architecture of RKIPC is as follows:



For the RKIPC application development guide, refer to `docs/en/Linux/ApplicationNote/Rockchip_Developer_Guide_Linux_RKIPC_EN.pdf`.

4.8 Rockchip Linux updateEngine Upgrade Guide

The Rockchip Linux platform supports two OTA upgrade solutions: **Recovery mode** and **Linux A/B mode**. Both are implemented through the updateEngine tool, whose source code is located at `external/recovery/update_engine`.

Upgrade Method	Advantages	Disadvantages	Reference Configuration
Recovery mode	Upgrade integrity is guaranteed; resumable after unexpected power loss	Must reboot into Recovery mode; cannot upgrade directly in normal system mode	Enabled by default; no extra configuration required
Linux A/B mode	No need to reboot into upgrade mode; automatic rollback on upgrade failure	Requires dual system partitions and more Flash space	`device/rockchip/.chips/rk3572/rockchip_rk3572_evb1_v10_ab_defconfig`

For device-side OTA upgrade, `--upgrade` must be used to specify the partitions to be upgraded except for the SD/USB automatic upgrade flow. `--upgrade=all` does not include `parameter` or `bootloader`. To upgrade GPT or IDBlock, explicitly specify the corresponding item. Mass-production OTA packages are not recommended to upgrade `parameter` or `bootloader` by default.

4.8.1 Recovery Mode OTA Upgrade

Run the following command in normal system mode to trigger the upgrade flow.

Online upgrade:

```
updateEngine --image_url=http://<server_ip>/update.img \
  --misc=update \
  --savepath=/userdata/update.img \
  --upgrade=boot,recovery,rootfs \
  --reboot &
```

Local upgrade:

```
updateEngine --image_url=/userdata/update.img \
  --misc=update \
  --savepath=/userdata/update.img \
  --upgrade=boot,recovery,rootfs \
  --reboot &
```

Upgrade flow: download firmware -> upgrade recovery partition -> reboot into Recovery mode -> upgrade target partitions -> reboot back to normal system.

To upgrade common partitions in the package except `userdata`, use `--upgrade=all,-userdata`. If the upgrade package is stored under `/userdata`, do not upgrade `userdata` in the same OTA operation.

4.8.2 Linux A/B Mode OTA Upgrade

Run the following command to generate the OTA firmware package:

```
./build.sh ota-updateimg
```

Online upgrade:

```
updateEngine --image_url=http://<server_ip>/update-ota.img \
  --update \
  --upgrade=boot,system \
  --reboot
```

Local upgrade:

```
updateEngine --image_url=/userdata/update-ota.img \
   --update \
   --upgrade=boot,system \
   --reboot
```

Upgrade flow: download firmware -> upgrade the inactive slot partitions -> mark inactive slot as next boot target -> reboot and switch slot.

In A/B systems, rootfs partitions must be named `system_a/system_b`, and the upgrade command uses `--upgrade=boot,system`. updateEngine automatically selects the target partitions in the inactive slot according to the current slot.

4.8.3 Differential Upgrade

The SDK provides the differential upgrade tool `tools/linux/Linux_Diff_Firmware/`, supporting differential updates for U-Boot, trust, kernel, and squashfs rootfs partitions to significantly reduce package size. Differential upgrade does not support ext2/ext4 filesystems, encrypted partitions, or A/B mode.

```
sudo apt install bsdiff
pip install ubi_reader
cd tools/linux/Linux_Diff_Firmware/
./mk-diff-ota.sh update_old.img update_new.img update_diff.img
```

The generated `update_diff.img` can be used in the same way as a normal OTA package. The old firmware must match the current firmware on the target device, and the free partition space on the target device must be sufficient for both the differential package and the new image data generated while applying the differential update.

4.8.4 SD Card Boot Upgrade

Use `tools/windows/SDDiskTool` to create an SD-card boot disk. After insertion and power-on, the device upgrades automatically. Use `update.img` for Recovery mode.

> **Note:** For SD-card upgrade in A/B mode, configure `RK_SDCARD_UPDATE=y` in defconfig and ensure that `recovery.img` has been built. After `./build.sh updateimg` is executed, the SDK generates `update-ab.img` and `update-sd.img`. Use `update-sd.img` to create the SD-card boot disk, then copy `update-ab.img` to the SD card root directory and rename it to `sdupdate.img`.

For detailed updateEngine upgrade information, refer to:

`docs/en/Linux/Recovery/Rockchip_Developer_Guide_Linux_Upgrade_EN.pdf`

4.9 Rockchip Linux ROS2 Software Development Guide

Rockchip Linux SDK is Buildroot-based and supports ROS2 cross-compilation from source. Jazzy and Humble LTS are currently supported, with Jazzy recommended. ROS2 cross-compilation is completed in an independent Docker environment, decoupled from the SDK. The compilation script, Dockerfile, and patches are all provided under `docs/Patches/ROS2/` in the SDK.

4.9.1 Build ROS2 Dependency Packages

Add ROS2 dependency configuration in Buildroot defconfig, then rebuild rootfs:

```
# Taking Jazzy as an example, append at the end of buildroot/configs/rockchip_rk3572_defconfig
#include "ros2/jazzy_dep.config"

./build.sh rootfs
```

4.9.2 Prepare Docker Environment

Install Docker and required tools:

```
sudo apt install docker.io python3-vcstool qemu-user-binfmt
sudo usermod -aG docker $USER && newgrp docker
```

After downloading ROS2 source and applying SDK patches, build the Docker image using the unified Dockerfile. Specify the ROS2 version via `--build-arg ROS_DISTRO`:

```
docker build --build-arg ROS_DISTRO=jazzy \
  --build-arg HOST_USER=$(whoami) --build-arg HOST_UID=$(id -u) --build-arg HOST_GID=$(id -g) \
  -t jazzy-ros2-builder -f rosdep.Dockerfile ./
```

4.9.3 Cross-Compile ROS2 in Docker

Mount the Buildroot output directory into Docker and run ROS2 cross-compilation. Artifacts are output to `target/opt/ros/`. Full compilation takes approximately 1–2 hours:

```
docker run -it --rm --privileged \
  -u $(id -u):$(id -g) \
  -v /etc/passwd:/etc/passwd:ro -v /etc/group:/etc/group:ro \
  --mount type=bind,source=<SDK>/buildroot/output/rockchip_rk3572/rockchip_rk3572,target=/buildroot \
  jazzy-ros2-builder:latest

./build_ros2.sh jazzy
```

4.9.4 Package rootfs and Verify

After exiting Docker, repack rootfs. After flashing, run validation commands on the board:

```
./build.sh rootfs

source /opt/ros/local_setup.sh
ros2 run demo_nodes_cpp talker &
ros2 run demo_nodes_cpp listener
```

> **Note:** Packages depending on Qt/X11, such as rviz, rqt, and turtlesim, are not supported for compilation and running in the Buildroot environment.

For detailed ROS2 compilation instructions, refer to:

`docs/en/Linux/ApplicationNote/Rockchip_Instruction_Linux_ROS/Rockchip_Instruction_Linux_ROS2_EN.pdf`

4.10 Rockchip IDC Software Development Guide

IDC (Image Distortion Correction) uses GPU parallel computing to efficiently correct image distortion. It currently depends on rockit and is intended for RKIPC scenarios. The following describes IDC demo build and run steps using `rockchip_rk3572_evb1_v10` Buildroot configuration as an example.

4.10.1 Build Configuration

Enable the following options in Buildroot configuration:

```
BR2_PACKAGE_COMMON_ALGORITHM=y
BR2_PACKAGE_SAMPLES=y
BR2_PACKAGE_SAMPLES_RK3572=y
```

Based on an existing successful Buildroot build, run the following commands in sequence to rebuild related modules:

```
make rockit-recreate
make common_algorithm-recreate
make samples-recreate
```

4.10.2 Run Example

On the target board, refer to `external/samples/simple_test/simple_cmd_rk3572.txt` for sample commands. A typical usage is:

```
simple_vi_gdc_vo -w 1920 -h 1088 -m 1 -M 3 -W 1 \
  -s /data/rkalg_idc_gpu_meshxy_step16x8.bin \
  -f /usr/lib/libmali.so -l 100
```

> **Note:** Before running, ensure `/data/rkalg_idc_gpu_meshxy_step16x8.bin` exists. This file varies by camera model and must be generated through calibration. The SDK provides one sample file for basic validation at `external/common_algorithm/misc/rkidc_config/rkalg_idc_gpu_meshxy_step16x8.bin`.

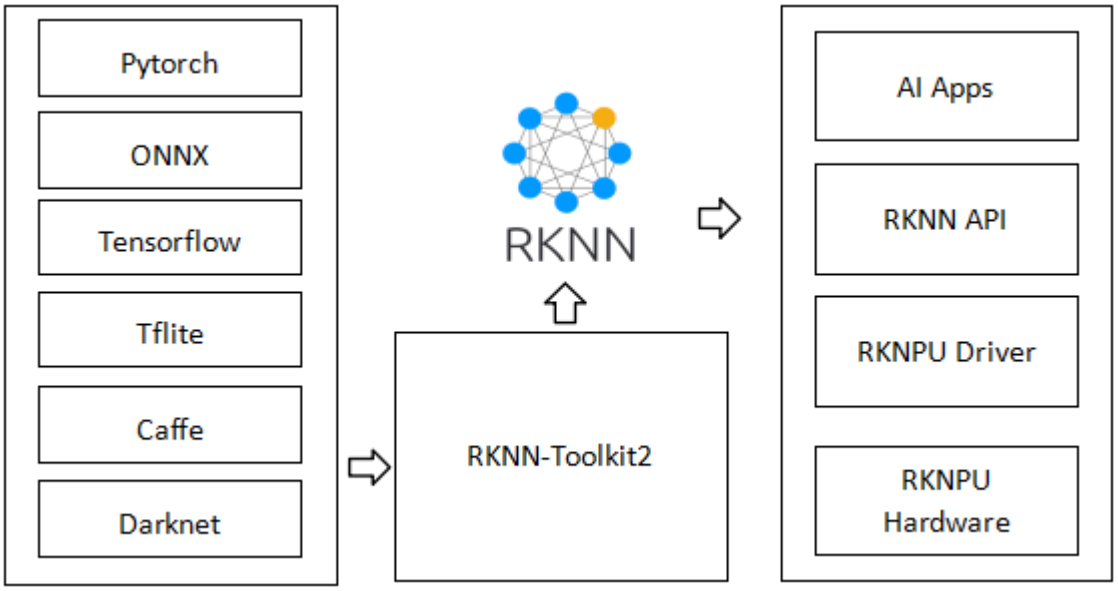
For IDC details, refer to:

`docs/en/Common/Rockchip_IDC_Developer_Guide_EN.pdf`

`docs/en/Common/Rockchip_IDC_Developer_Tutorial_EN.pdf`

4.11 RKNN Development Guide

The RKNN software stack helps users quickly deploy AI models to Rockchip chips. The overall framework is as follows:



Before using the RKNPU, users first run a PC-side tool (RKNN-Toolkit2 or RKNN3-Toolkit) on a computer to convert a trained model into the RKNN format, then perform inference on the development board through the C/C++ or Python API provided by the board-side Runtime.

Different chips adopt different generations of the NPU software stack. Select the toolchain and documentation for the corresponding platform according to the table below:

NPU Stack	Applicable Chips	PC-side Tool	Board-side Runtime	Kernel Driver
RKNPU2	RK3576, RK3588, etc.	RKNN-Toolkit2 (`external/rknn-toolkit2`)	`external/rknpu2`	`drivers/rknpu`
RKNPU3	RK3572	RKNN3-Toolkit (`external/rknn3-toolkit`)	`external/rknn3-runtime`	`drivers/rknpu`

4.11.1 RKNPU2 Platforms (RK3576, RK3588, etc.)

RKNN-Toolkit2

RKNN-Toolkit2 is a development kit for RKNN model generation and performance evaluation on a PC; RKNN-Toolkit-Lite2 provides a Python programming interface for the Rockchip NPU platform to help users deploy RKNN models and accelerate AI applications. The kit is located in the `external/rknn-toolkit2` directory and implements model conversion, optimization, quantization, inference, performance evaluation, and accuracy analysis.

The basic functions are as follows:

Function	Description
Model Conversion	Supports floating-point models of Pytorch / TensorFlow / TFLite / ONNX / Caffe / Darknet Supports quantization-aware training (QAT) models of Pytorch / TensorFlow / TFLite Supports dynamic-input models (dynamic/native dynamic) Supports large models
Model Optimization	Constant folding / OP correction / OP Fuse&Convert / weight sparsification / model pruning
Model Quantization	Quantization types: asymmetric i8 / fp16 Layer / Channel quantization; Normal / KL / MMSE quantization algorithms Mixed quantization to balance performance and accuracy
Model Inference	Supports model inference on PC via simulator Supports inference on the NPU hardware platform (connected-board inference) Supports batch inference and multi-input models
Model Evaluation	Supports performance and memory evaluation of models on the NPU hardware platform
Accuracy Analysis	Supports quantization accuracy analysis (simulator / NPU)
Additional Features	Supports version / device query, etc.

RKNN Runtime

The RKNN Runtime development documentation is located in the `external/rknpu2` directory. It is used to infer the rknn models generated by RKNN-Toolkit2 and provides a C/C++ programming interface for the Rockchip NPU platform. For detailed usage, refer to the documents in the `external/rknpu2/doc/` directory:

```
├── 01_Rockchip_RKNPU_Quick_Start_RKNN_SDK_V2.3.2_EN.pdf
├── 02_Rockchip_RKNPU_User_Guide_RKNN_SDK_V2.3.2_EN.pdf
├── 03_Rockchip_RKNPU_API_Reference_RKNN_Toolkit2_V2.3.2_EN.pdf
├── 04_Rockchip_RKNPU_API_Reference_RKNNRT_V2.3.2_EN.pdf
├── 05_RKNN_Compiler_Support_Operator_List_V2.3.2.pdf
```

RKNN Driver

The RKNN kernel driver interacts with the NPU hardware and is integrated into the Rockchip kernel code under the `drivers/rknpu` directory.

RKNN LLM

RKNN LLM is Rockchip's inference framework optimized for large language model (LLM) scenarios. The source code is located in `external/rknn-llm` and is used to efficiently deploy and run large pre-trained language models (such as LLaMA, ChatGLM, Qwen, etc.) on the Rockchip NPU hardware platform. For detailed usage, refer to the documents in the `external/rknn-llm/doc/` directory:

```
├── Rockchip_RKLLM_SDK_EN_1.3.0.pdf
```

4.11.2 RKNPU3 Platforms (RK3572)

RKNN3-Toolkit

The RKNN3 SDK provides a complete software stack for deploying AI models to RK3572. RKNN3-Toolkit is the PC-side development kit located in the `external/rknn3-toolkit` directory and supports model conversion, inference, and performance evaluation. For detailed usage, refer to the documents in the `external/rknn3-toolkit/doc/` directory:

```
├── 01_Rockchip_RKNPU3_Quick_Start_RKNN3_SDK_V1.0.4_EN.pdf
├── 02_Rockchip_RKNPU3_User_Guide_RKNN3_SDK_V1.0.4_EN.pdf
├── 03_Rockchip_RKNPU3_API_Reference_RKNN3_Toolkit_V1.0.4_EN.pdf
├── 04_Rockchip_RKNPU3_API_Reference_RKNNRT3_V1.0.4_EN.pdf
├── 05_Rockchip_User_Guide_RKNN3_Toolkit_Lite_V1.0.4_EN.pdf
```

RKNN3 Runtime

RKNN3 Runtime is the board-side runtime library located in the `external/rknn3-runtime` directory. It provides a C/C++ programming interface for deploying RKNN3 models and accelerating AI applications.

RKNN3 Driver

The RKNN3 kernel driver is also integrated into the Rockchip kernel code under the `drivers/rknpu` directory.

RKNN3 Model Zoo

RKNN3 Model Zoo is located in the `external/rknn3_model_zoo` directory and provides reference implementations for converting and deploying various models such as CNN / LLM / VLM.

RKLLM3

The LLM inference service for RK3572 is located in `external/rknn3-runtime/rkllm3-server` and works together with the RKNN3 Model Zoo to deploy large language models on RKNPU3 platforms.

4.12 RKAI Development Guide

RKAI provides validation programs and application development interfaces for large language models (LLMs) and vision language models (VLMs). The source code is located in `app/rkai`. RKAI supports both the RKNN3 (RKNPU3) and RKLLM/RKNPU2 backends; for the NPU stack selection and the corresponding chip platforms, refer to the "RKNN Development Guide" above. The following examples use the RKNN3 backend supported by RK3572. Other RKNPU2 platforms use the RKLLM/RKNPU2 backend. Enablement configuration is subject to the corresponding platform configuration in the Linux6.12 SDK.

4.12.1 Build Configuration

Enable RKAI in the Buildroot configuration and rebuild the rootfs:

```
#include "ai/rkai.config"
./build.sh rootfs
```

4.12.2 Model Files

An LLM adapted for RKNN3 requires an `.rknn` model file and the corresponding `.weight`, `.tokenizer.gguf`, and `.embed.bin` files. A VLM also requires the corresponding Vision `.rknn` and `.weight` files.

For the RKLLM/RKNPU2 backend, an LLM uses an `.rkllm` model. A VLM uses a Vision `.rknn` model and an LLM `.rkllm` model. The configuration file is `rkai_cfg.ini`.

For RKNN3 model conversion and deployment examples, refer to:

- RKNN3-Toolkit: https://github.com/airockchip/rknn3-toolkit
- RKNN3 Model Zoo: https://github.com/airockchip/rknn3-model-zoo

For RKLLM/RKNPU2 model conversion tools, refer to:

- Use RKLLM-Toolkit to convert LLM models: https://github.com/airockchip/rknn-llm
- Use RKNN-Toolkit2 to convert Vision models: https://github.com/airockchip/rknn-toolkit2

4.12.3 LLM Run Examples

RKNN3 backend:

```
rkai_LLM_Demo \
  --llm_model /userdata/<llm_model_dir>/ \
  --config /usr/share/app_conf/config/rkai_cfg_rk3572.ini
```

RKLLM/RKNPU2 backend:

```
rkai_LLM_Demo \
  --llm_model /userdata/<llm_model>.rkllm \
  --config /usr/share/app_conf/config/rkai_cfg.ini
```

4.12.4 VLM Run Examples

RKNN3 backend:

```
rkai_VLM_Demo \
  --vision_model /userdata/<vlm_model_dir>/ \
  --llm_model /userdata/<vlm_model_dir>/ \
  --image_path /userdata/<vlm_model_dir>/<input_image>.jpg \
  --config /usr/share/app_conf/config/rkai_cfg_rk3572.ini
```

RKLLM/RKNPU2 backend:

```
rkai_VLM_Demo \
  --vision_model /userdata/<vision_model>.rknn \
  --llm_model /userdata/<llm_model>.rkllm \
  --image_path /userdata/<input_image>.jpg \
  --config /usr/share/app_conf/config/rkai_cfg.ini
```

For RKAI details, refer to:

`docs/en/Linux/RKAI/Rockchip_Developer_Guide_Linux_RKAI_EN.pdf`

4.13 Software Update Log

- Software release version upgrades can be viewed through the project XML. The specific method is as follows:

```
.repo/manifests$ realpath rockchip_linux6.12_release.xml
```

For example: The printed version number is v1.0.0, and the update time is 20260620. The corresponding path is as follows:

```
<SDK>/.repo/manifests/release/rockchip_linux6.12_release_v1.0.0_20260620.xml
```

- The content of the software release version upgrades can be viewed through the project text, refer to the project directory:

```
<SDK>/.repo/manifests/rockchip_linux6.12_SDK_Note.md
or
<SDK>/docs/en/Release/Linux6.12/rockchip_linux6.12_SDK_Note.md
```

- The board can obtain version information in the following way:

```
buildroot:/# cat /etc/os-release
NAME=Buildroot
VERSION=linux-6.12-stan-rkr1
ID=buildroot
VERSION_ID=2026.02.2
PRETTY_NAME="Buildroot 2026.02.2"
ID_LIKE="buildroot"
RK_BUILD_INFO="<build_user> Sat Jun 20 14:35:23 CST 2026 - rockchip_<chip>"
```

5. SDK Project Directory Overview

The SDK project directory includes directories such as buildroot, debian, yocto, app, kernel, u-boot, device, docs, external, prebuilts, etc. It uses a manifest to manage the repository and the repo tool to manage each directory or its corresponding git projects.

- app: Contains upper-layer application APPs, mainly some application demos.
- buildroot: The root file system developed based on Buildroot (2026.02).
- debian: The root file system developed based on Debian trixie (13).
- device/rockchip: Contains chip-level board configurations as well as scripts and files for SDK compilation and firmware packaging.
- docs: Contains general development guidance documents, chip platform-related documents, Linux system development-related documents, and other reference documents.
- external: Contains third-party related repositories, including display, audio and video, camera, network, security, etc.
- kernel: Contains the code for Kernel development.
- output: Contains information about each firmware generation, compilation information, XML, host environment, etc.
- prebuilts: Contains cross-compilation toolchains.
- rkbin: Contains Rockchip-related binaries and tools.
- rockdev: Contains compiled output firmware, actually a soft link to `output/firmware`.
- tools: Contains commonly used tools for Linux and Windows operating systems.
- u-boot: Contains U-Boot code developed based on the v5 version.
- yocto: The root file system developed based on Yocto 6.0.

6. Introduction to Cross-Compilation Toolchain for SDK

Considering that the Rockchip Linux SDK is currently compiled only in the Linux PC environment, we have also provided the cross-compilation toolchain for Linux only. The prebuilt toolchains in the `prebuilts` directory are used for U-Boot and Kernel compilation. Each Rootfs requires its corresponding toolchain or a third-party toolchain. (The following examples use RK3572 as an example.)

6.1 U-Boot and Kernel Compilation Toolchain

The SDK prebuilts directory contains cross-compilation tools currently used for U-Boot and Kernel compilation, as follows:

Directory	Description
prebuilts/gcc/linux-x86/aarch64/arm-gnu-toolchain-14.3.rel1-x86_64-aarch64-none-linux-gnu	GCC ARM 14.3.rel1 64-bit toolchain

Directory	Description
prebuilts/gcc/linux-x86/arm/arm-gnu-toolchain-14.3.rel1-x86_64-arm-none-linux-gnueabihf	GCC ARM 14.3.rel1 32-bit toolchain

You can download the toolchain from the following address:

Click here

6.2 Buildroot Toolchain

6.2.1 Configuring the Compilation Environment

To compile individual modules or third-party applications, a cross-compilation environment must be set up. For instance, for the RK3572, the cross-compilation tools are located in the `buildroot/output/<RK_DEFCONFIG>/rockchip_rk3572/host/usr` directory, where `<RK_DEFCONFIG>` indicates the currently selected Buildroot defconfig directory name. It is necessary to set the bin/ directory of the tools and the `aarch64-buildroot-linux-gnu/bin/` directory as environment variables. Execute the script for automatic configuration of the environment variables in the top-level directory:

```
source buildroot/envsetup.sh rockchip_rk3572
```

Enter the command to check:

```
cd buildroot/output/<RK_DEFCONFIG>/rockchip_rk3572/host/usr/bin
./aarch64-linux-gcc --version
```

At this point, the following information will be printed:

```
aarch64-linux-gcc.br_real (Buildroot -gf133d0a00d) 14.3.0
```

6.2.2 Packaging Toolchain

Buildroot supports the packaging of the built-in toolchain into a compressed archive for standalone compilation by third-party applications. For detailed information on how to package the toolchain, please refer to the Buildroot official documentation:

```
buildroot/docs/manual/using-buildroot-toolchain.adoc
```

In the SDK, you can directly run the following command to generate the toolchain package:

```
./build.sh bmake:sdk
```

The generated toolchain package is located in the `buildroot/output/*/images/` directory, named `aarch64-buildroot-linux-gnu_sdk-buildroot.tar.gz`, for users who have the need. After extraction, the path to `gcc` will be:

```
./aarch64-buildroot-linux-gnu_sdk-buildroot/bin/aarch64-buildroot-linux-gnu-gcc
```

6.3 Debian Toolchain

Use Docker, gcc, or `dpkg-buildpackage` on the host machine for the relevant compilation.

6.4 Yocto Toolchain

Refer to the following:

- Building your own recipes from first principles
- New Recipe

7. SDK Compilation Instructions

The SDK can be configured and compiled for specific functionalities using `make` or `./build.sh` with target arguments. For detailed instructions, refer to the compilation guide in `device/rockchip/common/README.md`.

To ensure a smooth update process for the SDK each time, it is recommended to clean the previous compilation artifacts before updating. This practice helps to avoid potential compatibility issues or compilation errors that may arise from using old compilation artifacts with a new version of the SDK. To clean these artifacts, simply run the command `./build.sh cleanall`.

(The following examples use RK3572 as an example.)

One-Click Compilation

Chip	Type	Reference Model	One-Click Build	rootfs Build	kernel Build	uboot Build
RK3572	Development Board	RK3572 EVB1	./build.sh lunch:rockchip_rk3572_evb1_v10_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb1-v10-linux.img -j4	./make.sh rk3572 --spl-new
RK3572	Development Board (A/B Partition OTA)	RK3572 EVB1	./build.sh lunch:rockchip_rk3572_evb1_v10_ab_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb1-v10-linux.img -j4	./make.sh rk3572 --spl-new
RK3572	Development Board	RK3572 EVB2	./build.sh lunch:rockchip_rk3572_evb2_v10_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb2-v10-linux.img -j4	./make.sh rk3572 --spl-new
RK3572	Mono IPC	RK3572 EVB1	./build.sh lunch:rockchip_rk3572_ipc_evb1_v10_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb1-v10-linux.img -j4	./make.sh rk3572 --spl-new
RK3572	Robot	RK3572 EVB1	./build.sh lunch:rockchip_rk3572_evb1_robot_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb1-v10-linux.img -j4	./make.sh rk3572 --spl-new

Chip	Type	Reference Model	One-Click Build	rootfs Build	kernel Build	uboot Build
RK3572	Mono UVC	RK3572 EVB1	./build.sh lunch:rockchip_rk3572_uvc_evb1_v10_defconfig && ./build.sh	./build.sh rootfs	make ARCH=arm64 rockchip_linux_defconfig rk3572.config; make ARCH=arm64 rk3572-evb1-v10-linux.img -j4	./make.sh rk3572 --spl-new

> **Note:**
>
> - The table above uses RK3572 as an example. For other chips, replace the chip name, defconfig, and DTS in the commands with those of the target chip, i.e. `./build.sh lunch:rockchip_<chip>_<board>_defconfig`, `<chip>.config`, `<chip>-<board>-linux.img`, `./make.sh <chip>`.
> - **Rootfs Compilation**:
>   The default is to compile the Buildroot system. If other systems are required, set the corresponding environment variables. For example,
>   To compile the Buildroot system: RK_ROOTFS_SYSTEM=buildroot ./build.sh
>   To compile the Debian system: RK_ROOTFS_SYSTEM=debian ./build.sh
>   To compile the Yocto system: RK_ROOTFS_SYSTEM=yocto ./build.sh
> - **Kernel and U-Boot Compilation**: It is necessary to specify the toolchain.
> - For example, the SDK's built-in 32-bit toolchain:
>   export CROSS_COMPILE=../prebuilts/gcc/linux-x86/arm/arm-gnu-toolchain-14.3.rel1-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-
> - For example, the SDK's built-in 64-bit toolchain:
>   export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/arm-gnu-toolchain-14.3.rel1-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-

7.1 Viewing SDK Compilation Commands

`make help`, for example:

```
$ make help
  menuconfig          - interactive curses-based configurator
  oldconfig           - resolve any unresolved symbols in .config
  syncconfig          - Same as oldconfig, but quietly, additionally update deps
  olddefconfig        - Same as syncconfig but sets new symbols to their default value
  savedefconfig       - Save current config to RK_DEFCONFIG (minimal config)
  ...
```

The actual execution of make is `./build.sh`

You can also compile related features by running `./build.sh <target>`, and you can view the specific compilation commands through `./build.sh help`.

```
./build.sh -h

############### Rockchip Linux SDK ###############

Manifest: rockchip_linux6.12.xml
GIT commit: "e36cd901 tools: Support retaining original rootfs tools"
```

```
Log colors: message notice warning error fatal

Usage: build.sh [OPTIONS]
Available options:
chip[:<chip>[:<config>]]              choose chip
defconfig[:<config>]                  choose defconfig
 *_defconfig                          switch to specified defconfig
    available defconfigs:
    rockchip_defconfig
    rockchip_rk3572_evb1_robot_defconfig
    rockchip_rk3572_evb1_v10_ab_defconfig
    rockchip_rk3572_evb1_v10_defconfig
    rockchip_rk3572_evb2_v10_defconfig
    rockchip_rk3572_evb3_v10_defconfig
    rockchip_rk3572_iotest_v10_defconfig
    rockchip_rk3572_ipc_evb1_v10_defconfig
    rockchip_rk3572_test1_v10_defconfig
    rockchip_rk3572_test2_v10_defconfig
    rockchip_rk3572_test3_v10_defconfig
    rockchip_rk3572_uvc_evb1_v10_defconfig
 olddefconfig                         resolve any unresolved symbols in .config
 savedefconfig                        save current config to defconfig
 menuconfig                           interactive curses-based configurator
config                               modify SDK defconfig
config-rootfs-type                   modify SDK rootfs type config
config-usb-gadget                    modify SDK USB gadget config
config-wifibt                        modify SDK Wi-Fi/BT config
config-rootfs-overlay                modify SDK rootfs overlay config
config-overlay                       alias of config-rootfs-overlay
config-post-rootfs                   modify SDK post rootfs config
config-extra-parts                   modify SDK extra partitions config
print-parts                          print partitions
list-parts                           alias of print-parts
edit-parts                           edit raw partitions
misc                                 pack misc image
kernel-5.10[:dry-run]                build kernel 5.10
kernel-6.1[:dry-run]                 build kernel 6.1
kernel-6.12[:dry-run]                build kernel 6.12
kernel[:dry-run]                     build kernel
recovery-kernel[:dry-run]            build kernel for recovery
kernel-modules[:<dst dir>:dry-run]   build kernel modules
modules[:<dst dir>:dry-run]          alias of kernel-modules
linux-headers[:<arch>:dry-run]       build linux-headers
kernel-config[:dry-run]              modify kernel defconfig
kconfig[:dry-run]                    alias of kernel-config
kernel-make[:<arg1>:<arg2>]          run kernel make
kmake[:<arg1>:<arg2>]                alias of kernel-make
wifibt[:<dst dir>[:<chip>]]          build Wifi/BT
amp                                  build and pack amp system
buildroot-config[:<config>]          modify buildroot defconfig
bconfig[:<config>]                   alias of buildroot-config
buildroot-make[:<arg1>:<arg2>]       run buildroot make
bmake[:<arg1>:<arg2>]                alias of buildroot-make
buildroot-sdk                        build the buildroot SDK tarball
```

```
bsdk                            alias of buildroot-sdk
edit-debian-packages              edit debian package list
rootfs[:<rootfs type>]            build default rootfs
buildroot                       build buildroot rootfs
yocto                           build yocto rootfs
debian                          build debian rootfs
recovery                        build recovery
security-createkeys               create keys for security
security-misc                   build misc with system encryption key
security-ramboot[:system_image]     build security ramboot
security-system[:system_image]      build security system
security-remote-sign              build remote signed image
loader[:dry-run]                  build loader (u-boot)
uboot[:dry-run]                   build u-boot
u-boot[:dry-run]                  alias of uboot
extra-parts                     pack extra partition images
firmware                        pack and check firmwares
edit-package-file                 edit package-file
edit-ota-package-file             edit package-file for OTA
updateimg                       build update image
ota-updateimg                     build update image for OTA
sd-updateimg                      build update image for SDcard
all                           build images
release[:<subdir>[:<name>]]       release images and build info
all-release[:<subdir>[:<name>]]     build and release images
shell                           setup a shell for developing
buildroot-shell                   setup a shell for buildroot developing
bshell                          alias of buildroot-shell
yocto-shell                       setup a shell for yocto developing
yshell                          alias of yocto-shell
cleanall                        cleanup all
clean-misc                        cleanup misc
clean-config                      cleanup config
clean-recovery                    cleanup recovery
clean-amp                         cleanup amp
clean-kernel                      cleanup kernel
clean-rootfs                      cleanup rootfs
clean-firmware                    cleanup firmware
clean-security                    cleanup security
clean-extra-parts                 cleanup extra-parts
clean-updateimg                   cleanup updateimg
clean-loader                      cleanup loader
post-rootfs <rootfs dir>            trigger post-rootfs hook scripts
help                            display this information

Default option is 'all'.
```

7.2 SDK Board-Level Configuration

Linux6.12 is a unified SDK covering chips such as RK3572, RK3576, and RK3588. Before building, you must first select the target chip, then select the corresponding board-level configuration (defconfig) for that chip. Chip selection and board-level configuration switching use `./build.sh chip` and `./build.sh lunch` respectively; their behaviors differ and are described separately below.

7.2.1 Chip Selection

`./build.sh chip` is used to select the target chip. It first lists the chips supported by the SDK for selection (Pick a chip); after a chip is selected, it then lists the board-level configurations under that chip for selection (Pick a defconfig):

```
./build.sh chip

################ Rockchip Linux SDK ################

Pick a chip:

1. rk3572
2. rk3576
3. rk3588
Which would you like? [1]:

Pick a defconfig:

1. rockchip_defconfig
2. rockchip_rk3572_evb1_robot_defconfig
3. rockchip_rk3572_evb1_v10_ab_defconfig
4. rockchip_rk3572_evb1_v10_defconfig
5. rockchip_rk3572_evb2_v10_defconfig
6. rockchip_rk3572_evb3_v10_defconfig
7. rockchip_rk3572_iotest_v10_defconfig
8. rockchip_rk3572_ipc_evb1_v10_defconfig
9. rockchip_rk3572_test1_v10_defconfig
10. rockchip_rk3572_test2_v10_defconfig
11. rockchip_rk3572_test3_v10_defconfig
12. rockchip_rk3572_uvc_evb1_v10_defconfig
Which would you like? [1]:
```

The defconfigs listed under Pick a defconfig belong to the selected chip; here rk3572 is selected as an example.

7.2.2 Board Configuration Selection

`./build.sh lunch` (or `make lunch`) is used to switch the board-level configuration under an **already-selected chip**. It only lists the board-level configurations of the current chip (Pick a defconfig) and does not select a chip:

```
./build.sh lunch

############### Rockchip Linux SDK ###############

Pick a defconfig:

1. rockchip_defconfig
2. rockchip_rk3572_evb1_robot_defconfig
3. rockchip_rk3572_evb1_v10_ab_defconfig
4. rockchip_rk3572_evb1_v10_defconfig
5. rockchip_rk3572_evb2_v10_defconfig
6. rockchip_rk3572_evb3_v10_defconfig
7. rockchip_rk3572_iotest_v10_defconfig
8. rockchip_rk3572_ipc_evb1_v10_defconfig
9. rockchip_rk3572_test1_v10_defconfig
10. rockchip_rk3572_test2_v10_defconfig
11. rockchip_rk3572_test3_v10_defconfig
12. rockchip_rk3572_uvc_evb1_v10_defconfig
Which would you like? [1]:
```

The following uses RK3572 as an example; its board configurations are located in the `<SDK>/device/rockchip/.chips/rk3572` directory:

Board Configuration	Description
rockchip_defconfig	Default configuration, symlinked to RK3572 EVB1 board configuration
rockchip_rk3572_evb1_robot_defconfig	Suitable for RK3572 EVB1 Robot board configuration
rockchip_rk3572_evb1_v10_ab_defconfig	Suitable for RK3572 board configuration requiring A/B upgrade
rockchip_rk3572_evb1_v10_defconfig	Suitable for RK3572 EVB1 AIoT board configuration
rockchip_rk3572_evb2_v10_defconfig	Suitable for RK3572 EVB2 AIoT board configuration
rockchip_rk3572_evb3_v10_defconfig	Suitable for RK3572 EVB3 board configuration
rockchip_rk3572_iotest_v10_defconfig	Suitable for RK3572 IO test board configuration
rockchip_rk3572_ipc_evb1_v10_defconfig	Suitable for RK3572 IPC EVB1 board configuration
rockchip_rk3572_test1_v10_defconfig	Suitable for RK3572 test1 board configuration
rockchip_rk3572_test2_v10_defconfig	Suitable for RK3572 test2 board configuration
rockchip_rk3572_test3_v10_defconfig	Suitable for RK3572 test3 board configuration
rockchip_rk3572_uvc_evb1_v10_defconfig	Suitable for RK3572 Mono UVC EVB1 board configuration

> Most customers receive the EVB1 board, which is suitable for general AIoT application scenarios.

The board configuration flow for RK3576 and RK3588 is the same; their defconfigs are located in the `<SDK>/device/rockchip/.chips/rk3576` and `<SDK>/device/rockchip/.chips/rk3588` directories respectively, following the same naming convention (`rockchip_<chip>_<board>_defconfig`). Use `./build.sh chip` to select the corresponding chip and then view and choose them.

Other feature configurations can be set through `make menuconfig` to configure related properties.

7.3 SDK Custom Configuration

The SDK can be configured through `make menuconfig` , and the main configurable components are as follows:

```
(rockchip_rk3572_evb1_v10_defconfig) Name of defconfig to save
[*] Rootfs (Buildroot|Debian|Yocto)  --->
[*] Loader (U-Boot)  --->
[ ] AMP (Asymmetric Multi-Processing System)
[*] Kernel (Embedded in an Android-style boot image)  --->
    Boot (Android-style boot image)  --->
[*] Recovery (based on Buildroot)  --->
[ ] Security feature (secureboot, encryption, verity, etc.)
    Extra partitions (oem, userdata, etc.)  --->
    Firmware (partition table, misc image, etc.)  --->
[*] Update (Rockchip update image)  --->
    *** Fastboot is not supported for this chip ***
    Others configurations  --->
```

- Rootfs: The Rootfs here represents the "root file system," where you can choose different root file system configurations such as Buildroot, Yocto, Debian, etc.
- Loader (U-Boot): This is the configuration for the bootloader, typically U-Boot, which is used to initialize the hardware and load the main operating system.
- AMP: A multi-core heterogeneous boot solution suitable for application scenarios requiring real-time performance.
- Kernel: Here you configure the kernel options to customize the Linux kernel according to your hardware and application needs.
- Boot: Here you configure the support format for the Boot partition.
- Recovery (based on Buildroot): This is the configuration for the recovery environment based on Buildroot, used for system recovery and upgrades.
- Security: Security features are enabled, including Secureboot methods, Optee storage methods, and burn-in keys.
- Extra partitions: Used to configure additional partitions.
- Firmware: Here you configure firmware-related options.
- Update (Rockchip update image): Used to configure options for the Rockchip complete firmware.
- Others configurations: Other additional configuration options.

The `make menuconfig` configuration interface provides a text-based user interface for selecting and configuring various options.
After the configuration is complete, use the `make savedefconfig` command to save these configurations, so that the customized compilation will proceed according to these settings.

Through the above config, you can choose different Rootfs/Loader/Kernel configurations for various customized compilations, allowing you to flexibly select and configure system components to meet specific needs.

7.4 Fully Automatic Compilation

Navigate to the project's root directory and execute the following command to automatically complete all compilations:

```
./build.sh all # Builds modules and packages basic firmware into output/firmware
./build.sh     # The default target is all and is equivalent to the command above

./build.sh updateimg # Generates the update.img upgrade package from basic firmware
```

The default is Buildroot, but you can specify a different rootfs by setting the environment variable `RK_ROOTFS_SYSTEM`. `RK_ROOTFS_SYSTEM` currently supports three systems: buildroot, debian, and yocto.

For example, to generate a Debian system, you can use the following commands:

```
export RK_ROOTFS_SYSTEM=debian
./build.sh
or
RK_ROOTFS_SYSTEM=debian ./build.sh
```

7.5 Module Compilation

7.5.1 U-Boot Compilation

```
./build.sh uboot
```

7.5.2 Kernel Compilation

- Method One

```
./build.sh kernel
```

- Method Two

```
cd kernel
export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/arm-gnu-toolchain-14.3.rel1-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig rk3572.config
make ARCH=arm64 rk3572-evb1-v10-linux.img -j4
```

> **Note:** Replace `rk3572-evb1-v10-linux` with the DTS name for the target hardware when required. The parallel job count after `-j` can be adjusted based on the number of host CPU cores.

7.5.3 Recovery Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of Recovery.

```
<SDK># ./build.sh recovery
```

After compilation, the recovery.img is generated in the Buildroot directory `output/<RK_DEFCONFIG>/rockchip_rk3572_recovery/images`.

> **Note:** The recovery.img is an image that includes the kernel, so every time the Kernel is modified, Recovery needs to be repackaged and generated. The method to repackage Recovery is as follows:

```
<SDK># source buildroot/envsetup.sh
<SDK># cd buildroot
buildroot# make recovery-reconfigure
buildroot# cd -
<SDK># ./build.sh recovery
```

> **Note:** Recovery is a non-essential feature, and some board-level configurations may not set it up.

7.5.4 Buildroot Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of the Rootfs:

```
./build.sh rootfs
```

After the compilation, different formats of the image are generated in the `output/<RK_DEFCONFIG>/rockchip_rk3572/images` directory under the Buildroot directory, with the default format being rootfs.ext4.

For specific details, refer to the Buildroot development documentation:

`<SDK>/docs/en/Linux/System/Rockchip_Developer_Guide_Buildroot_EN.pdf`

7.5.4.1 Buildroot Module Compilation

Set up configurations for different chips and target features via `source buildroot/envsetup.sh`

```
$ source buildroot/envsetup.sh
Top of tree: RK3572

You're building on Linux
Lunch menu...pick a combo:

78. rockchip_rk3572
79. rockchip_rk3572_ab
80. rockchip_rk3572_ipc
81. rockchip_rk3572_pcba
82. rockchip_rk3572_ramboot
83. rockchip_rk3572_recovery
84. rockchip_rk3572_robot
85. rockchip_rk3572_uvc
...

Which would you like? [1]:
```

The default selection is 78, `rockchip_rk3572`. Then proceed to the RK3572's Buildroot directory to start compiling the relevant modules.

The `rockchip_rk3572_recovery` is used for compiling the Recovery module.

For example, to compile the rockchip-test module, the common compilation commands are as follows:

Navigate to the buildroot directory

```
<SDK># cd buildroot
```

- Delete and recompile rockchip-test

```
buildroot# make rockchip-test-recreate
```

- Rebuild rockchip-test

```
buildroot# make rockchip-test-rebuild
```

- Delete rockchip-test

```
buildroot# make rockchip-test-dirclean
or
buildroot# rm -rf output/rockchip_rk3572/build/rockchip-test-master/
```

7.5.5 Debian Compilation

```
./build.sh debian
```

After compilation, the `rockchip-rootfs.img` is generated in the `debian` directory.

> **Note: It is necessary to install the relevant dependency packages in advance.**

```
sudo apt-get install binfmt-support qemu-user-static live-build
sudo dpkg -i ubuntu-build-service/packages/*
sudo apt-get install -f
```

For specific details, please refer to the Debian development documentation:

`<SDK>/docs/en/Linux/System/Rockchip_Developer_Guide_Debian_EN.pdf`

7.5.6 Yocto Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of the Rootfs:

```
./build.sh yocto
```

After compilation, the rootfs.img is generated in the directory yocto/build/latest.

The default user login is root. For more information on Yocto, please refer to the Rockchip Wiki.

FAQ:

- If you encounter the following issue during the compilation:

```
Please use a locale setting which supports UTF-8 (such as LANG=en_US.UTF-8).
Python can't change the filesystem locale after loading so we need a UTF-8
when Python starts or things won't work.
```

Solution:

```
locale-gen en_US.UTF-8
export LANG=en_US.UTF-8 LANGUAGE=en_US:en LC_ALL=en_US.UTF-8
```

Or refer to setup-locale-python3

- If you encounter git permission issues that cause compilation errors.

Due to the addition of the CVE-2022-39253 security detection patch in the newer version of git, if an older version of Yocto is required, poky must include the following to be fixed:

```
commit ac3eb2418aa91e85c39560913c1ddfd2134555ba
Author: Robert Yang <liezhi.yang@windriver.com>
Date:   Fri Mar 24 00:09:02 2023 -0700

    bitbake: fetch/git: Fix local clone url to make it work with repo

    The "git clone /path/to/git/objects_symlink" couldn't work after the following
    change:

    https://github.com/git/git/commit/6f054f9fb3a501c35b55c65e547a244f14c38d56
```

Alternatively, you can also downgrade the PC's git version to V2.38 or an earlier version, for example:

```
$ sudo apt update && sudo apt install -y libcurl4-gnutls-dev

git clone https://gitee.com/mirrors/git.git --depth 1 -b v2.38.0
cd git
make git -j8
make install
sudo install -m 0755 git /usr/bin/git
```

7.6 Firmware Packaging

After compiling the various components such as Kernel, U-Boot, Recovery, and Rootfs, navigate to the root directory of the project and execute the following command to automatically complete the packaging of all firmware into the `output/firmware` directory:

Firmware Generation:

```
./build.sh firmware
```

8. Flashing Instructions

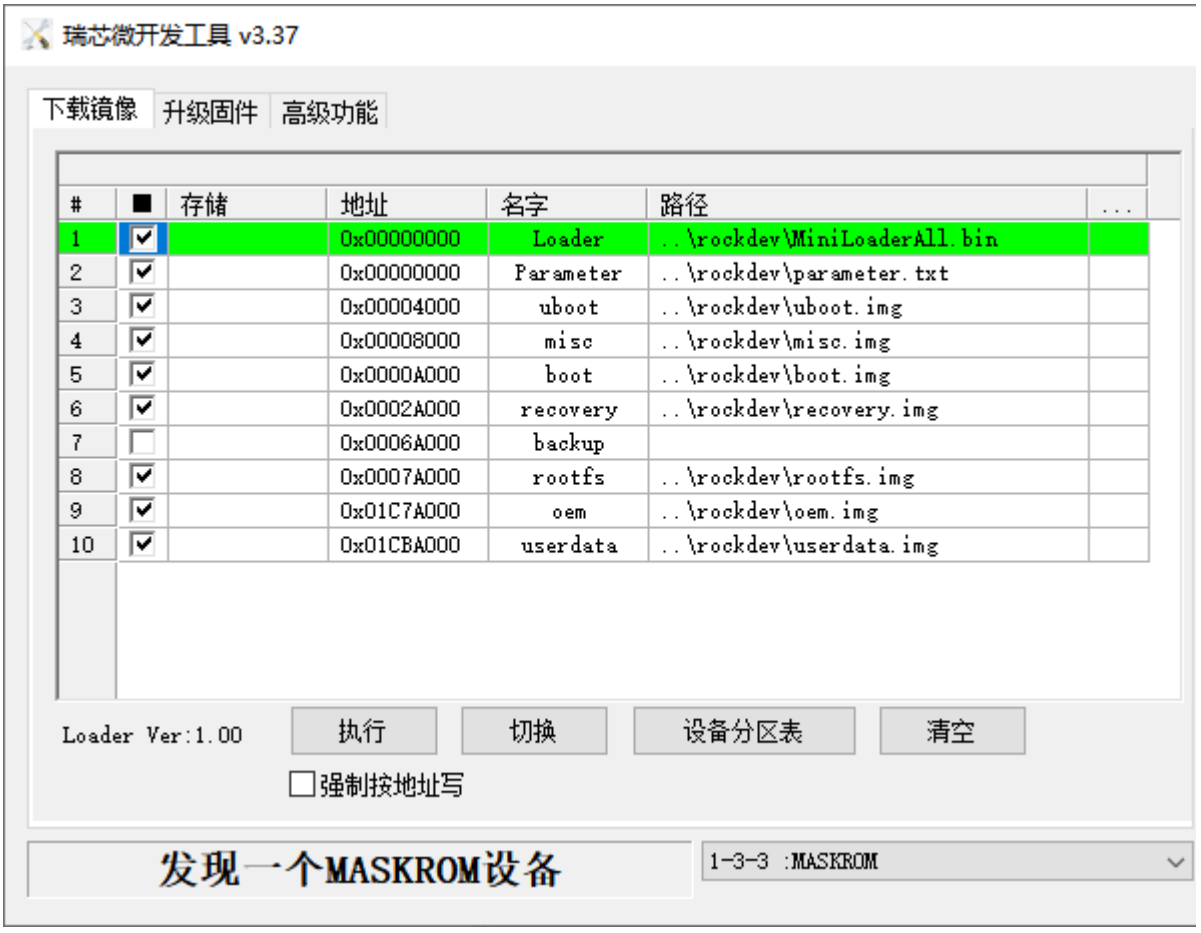
8.1 Windows Flashing Instructions

The SDK provides a Windows flashing tool (the tool version must be V3.28 or above), located in the root directory of the project:

```
tools/
  ├─── windows/RKDevTool
```

As shown in the figure below, after compiling the corresponding firmware, the device needs to enter the MASKROM or BootROM flashing mode. Connect the USB download cable, hold the "MASKROM" button, press and release the reset button "RST", then release the "MASKROM" button to enter MASKROM mode. Load the compiled firmware from the corresponding path and click "Execute" to flash. Alternatively, hold

the "recovery" button, press and release the reset button "RST", then release the "recovery" button to enter the loader mode for flashing. Below are the partition offsets for MASKROM mode and the flashing files.
(Note: On the Windows PC, the tool must be run with administrator privileges.)



Note: Before flashing, the latest USB driver must be installed. For more details on the driver, see:

```
<SDK>/tools/windows/DriverAssitant_v5.14.zip
```

8.2 Linux Flashing Instructions

The flashing tool for Linux is located in the `tools/linux` directory (the Linux_Upgrade_Tool version must be V2.36 or above). Please ensure that your board is connected to MASKROM/loader rockusb. For example, if the firmware compiled is in the `rockdev` directory, the upgrade commands are as follows:

```
sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rockdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd
```

Or upgrade the complete firmware after packaging:

```
sudo ./upgrade_tool uf rockdev/update.img
```

Or upgrade in the root directory when the machine is running in MASKROM state:

```
./rkflash.sh
```

8.3 System Partition Description

Default Partition Description (Below is the RK3572 EVB Partition Reference)

Number	Start	End	Size	Name
1	8389kB	16.8MB	8389kB	uboot
2	16.8MB	21.0MB	4194kB	misc
3	21.0MB	88.1MB	67.1MB	boot
4	88.1MB	222MB	134MB	recovery
5	222MB	256MB	33.6MB	backup
6	256MB	15.3GB	15.0GB	rootfs
7	15.3GB	15.4GB	134MB	oem
8	15.4GB	62.5GB	47.1GB	userdata

- uboot Partition: For the uboot.img compiled from uboot.
- misc Partition: For misc.img, used by recovery.
- boot Partition: For boot.img compiled from the kernel.
- recovery Partition: For recovery.img compiled by recovery.
- backup Partition: Reserved, not in use at the moment.
- rootfs Partition: For rootfs.img compiled by buildroot, debian, or yocto.
- oem Partition: For manufacturers to use, storing manufacturer's apps or data. Mounted at the /oem directory.
- userdata Partition: For apps to temporarily generate files or for end users, mounted under the /userdata directory.